Lab 2: Logistic Regression, Decision Trees, and Support Vector Machines

Arjun Aggarwal
Roll Number: 2024AIB1289
Course: AI211

Contents

1 Introduction **3**

2 Task 1: Experimental Analysis of Logistic Regression Variants **3**
2.1 Dataset Preparation and Ordinal Structure Exploration (Part 1a) 3
2.1.1 Data Loading and Preprocessing 3
2.1.2 Class Distribution and Overlap Analysis 4
2.2 Linear Classifier (Part 1b) – Multiclass Perceptron 6
2.2.1 Mathematical Formulation and Code Implementation 6
2.2.2 Results and Observations 6
2.3 One-vs-Rest Logistic Regression (Part 2) 8
2.3.1 Model Architecture and Mathematics 8
2.3.2 Training and Results 9
2.3.3 Observations from the Confusion Matrix 9
2.4 Effect of Regularization: L1 vs L2 (Part 3) 10
2.4.1 Mathematical Formulation 10
2.4.2 Results and Observations 11
2.5 Ordinal Logistic Regression (Part 4) 12
2.5.1 Architecture and Mathematical Derivation 12
2.5.2 Results and Observations 13
2.6 Summary and Comparison of Models (Part 5) 14

3 Task 2: Decision Trees Classification **16**
3.1 Dataset Understanding 16
3.2 Training a Decision Tree with Entropy (Part 2a) 16
3.2.1 Mathematical Formulation: Information Gain with Entropy 16
3.2.2 Tree Visualization and Structure 17
3.2.3 Evaluation Metrics 18
3.3 Entropy vs. Gini Index (Part 3) 18
3.3.1 Gini Impurity 18
3.3.2 Comparison 19

3.4 Role of Max Depth (Part 4) . . . 20
3.4.1 Experimental Setup . . . 20
3.4.2 Results and Discussion . . . 20
3.5 Random Forest Classifier (Part 5) . . . 21
3.5.1 Architecture of the Random Forest . . . 21
3.5.2 Comparison: Single Tree vs. Random Forest . . . 22
3.5.3 Explanation of Bagging's Effect . . . 23

4 Task 3: Support Vector Machine **24**
4.1 OVO Linear SVM Implementation and Hyperparameter Tuning (Part 1) . . . 24
4.1.1 Linear SVM: Mathematical Foundation and Code . . . 24
4.1.2 Hyperparameter Tuning for C . . . 24
4.2 Decision Boundary Visualization (Part 2) . . . 25
4.3 Analysis of Parameter C (Part 3) . . . 25
4.3.1 Impact on Support Vectors and Margin Violations . . . 25
4.4 Kernel SVM Analysis (Part 4) . . . 26
4.4.1 Kernel Trick and Implementation . . . 26
4.4.2 Comparison of Kernels . . . 27
4.5 Evaluation Metrics (Part 5) . . . 28

1 Introduction

The objectives are:

1. To implement and dissect various forms of Logistic Regression: linear classifier (perceptron), one-vs-rest (OvR), regularized variants (L1/L2), and ordinal logistic regression.

2. To build Decision Trees from scratch, investigate splitting criteria (Entropy vs. Gini), analyze the bias-variance tradeoff via depth, and construct a Random Forest ensemble.

3. To develop Support Vector Machines (SVMs) using a One-vs-One (OVO) strategy, tune hyperparameters, visualize decision boundaries, and explore kernel methods (Polynomial and RBF).

2 Task 1: Experimental Analysis of Logistic Regression Variants

2.1 Dataset Preparation and Ordinal Structure Exploration (Part 1a)

2.1.1 Data Loading and Preprocessing

The Car Evaluation dataset (`https://archive.ics.uci.edu/ml/datasets/car+evaluation`) consists of 1,728 instances described by six categorical attributes. The preprocessing steps, as implemented in the code, are critical for converting this non-numeric data into a form suitable for linear models.

1. **Loading:** The data is read into a Pandas DataFrame with predefined column names.

2. **Ordinal Target Encoding:** The original target labels (‘unacc‘, ‘acc‘, ‘good‘, ‘vgood‘) are ordinal. As per the lab instructions, they are mapped to three ordered classes:

   - `Low` ← `unacc` (encoded as label 1)
   - `Medium` ← `acc` (encoded as label 2)
   - `High` ← `good` and `vgood` (encoded as label 3)

   This mapping is performed by two dictionaries: `mapping_a` (for string labels) and `mapping_b` (for numeric labels). The final label vector `y` contains values in $\{1, 2, 3\}$.

3. **Feature Encoding (One-Hot Encoding):** Since all six features are categorical, a one-hot encoding scheme is applied using `pd.get_dummies()`. This creates a binary vector for each feature. For a feature with k possible values, this results in k binary columns, exactly one of which is ’1’ for a given sample.

   - `buying`: vhigh, high, med, low → 4 columns
   - `maint`: vhigh, high, med, low → 4 columns

- `doors`: 2, 3, 4, 5more $\rightarrow$ 4 columns
- `persons`: 2, 4, more $\rightarrow$ 3 columns
- `lug_boot`: small, med, big $\rightarrow$ 3 columns
- `safety`: low, med, high $\rightarrow$ 3 columns

The resulting feature matrix `X` has a total of $4+4+4+3+3+3=21$ columns. This is evident in the code's output showing `X.shape` as (1728, 21).

2.1.2 Class Distribution and Overlap Analysis

A crucial first step in any classification task is understanding the target variable.

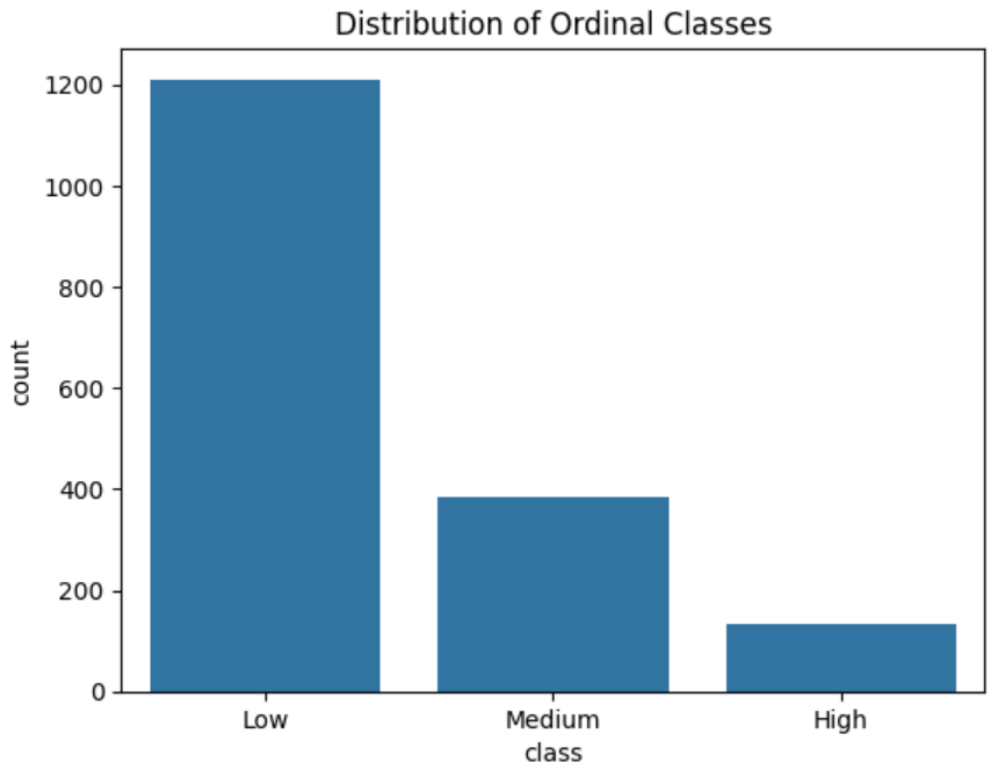


Figure 1: Distribution of the three ordinal classes after mapping.

Observations:

1. **Class Imbalance:** The countplot (Figure 1) reveals a severe class imbalance.

   - Class 1 (Low): 1210 samples (70.0%)
   - Class 2 (Medium): 384 samples (22.2%)
   - Class 3 (High): 134 samples (7.8%)

   This imbalance is a significant characteristic of the dataset. A naive classifier that always predicts the majority class ("Low") would achieve an accuracy of 70%. Any useful model must outperform this baseline. The imbalance can bias models towards the majority class, making it harder to correctly classify the minority "High" class.

2. **Feature Overlap Between Adjacent Classes:** To assess the separability of classes, we analyze bar plots of feature values for each class (Figure 2). The visual comparison between `Low` vs. `Medium` and `Medium` vs. `High` is key.

   - **Low vs. Medium:** For features like `buying` and `maint`, the distribution of categories (e.g., low, med, high, vhigh) is very similar between these two classes. This suggests a high degree of overlap, implying that distinguishing between a "Low" and "Medium" evaluation is challenging based on these features alone.
   - **Medium vs. High:** While the distributions are still somewhat similar, there are often more discernible differences. For instance, a higher proportion of "High" evaluations might have higher `safety` or larger `lug_boot`. However, the overlap remains significant.

   The significant overlap, particularly between adjacent ordinal classes, is the primary motivation for exploring ordinal logistic regression, which is designed to leverage this inherent ordering. Standard multi-class models (like OvR) treat these as independent categories, discarding the ordinal information and potentially leading to illogical "extreme errors" (e.g., predicting "High" for a "Low" sample).

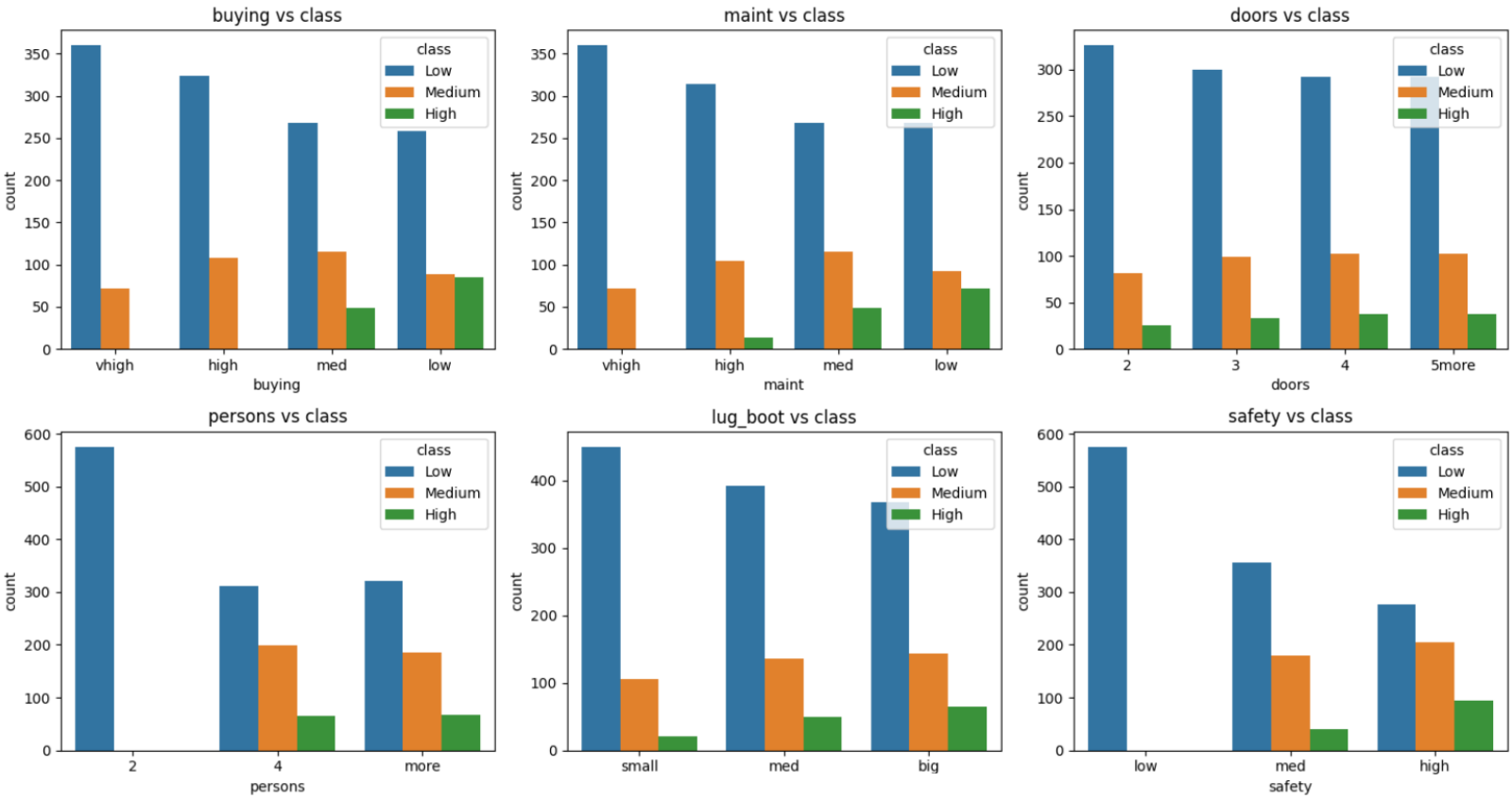


Figure 2: Bar plots of feature categories for each class. Similar bar shapes between adjacent classes indicate overlap.

2.2 Linear Classifier (Part 1b) – Multiclass Perceptron

2.2.1 Mathematical Formulation and Code Implementation

The multiclass perceptron is an extension of the classic binary perceptron. The code implements a batch update version.

Definition 2.1 (Multiclass Perceptron Update Rule)**.** Given an input vector $\mathbf{x} \in \mathbb{R}^{21}$ and its true class $y \in \{1, 2, 3\}$, the classifier maintains a weight vector $\mathbf{w}_k \in \mathbb{R}^{21}$ and a bias $b_k \in \mathbb{R}$ for each class k. The score for class k is computed as:

$$s_k(\mathbf{x}) = \mathbf{w}_k^\top \mathbf{x} + b_k.$$

The predicted class is $\hat{y} = \arg\max_k s_k(\mathbf{x})$.

If the prediction is incorrect ($\hat{y} \neq y$), the weights and biases are updated as:

$$\begin{aligned} \mathbf{w}_y &\leftarrow \mathbf{w}_y + \eta\,\mathbf{x} \\ \mathbf{w}_{\hat{y}} &\leftarrow \mathbf{w}_{\hat{y}} - \eta\,\mathbf{x} \\ b_y &\leftarrow b_y + \eta \\ b_{\hat{y}} &\leftarrow b_{\hat{y}} - \eta \end{aligned}$$

where η is the learning rate. This update reinforces the correct class and penalizes the incorrect one.

In the code, the algorithm is implemented as follows:

- `weights` is a 2D NumPy array of shape `(features, classes)` = (21, 3).
- `bias` is an array of shape `(classes,)`.
- `eta = 0.05` and `epochs = 100`.
- The inner loop iterates over all training samples. For each sample, it computes `scores = np.dot(X[i], weights) + bias` and finds `prediction = np.argmax(scores) + 1`.
- If `prediction` $\neq$ `target`, the corresponding rows of `weights` and elements of `bias` are updated according to the rule above.

2.2.2 Results and Observations

- **Overall Training Accuracy:** `90.2199074074074%`
- **Visualization:** A scatter plot (Figure 3) using the `buying` and `maint` features (with jitter added to avoid overlapping points) shows the decision regions. Misclassified samples are marked in red.

Detailed Observations:

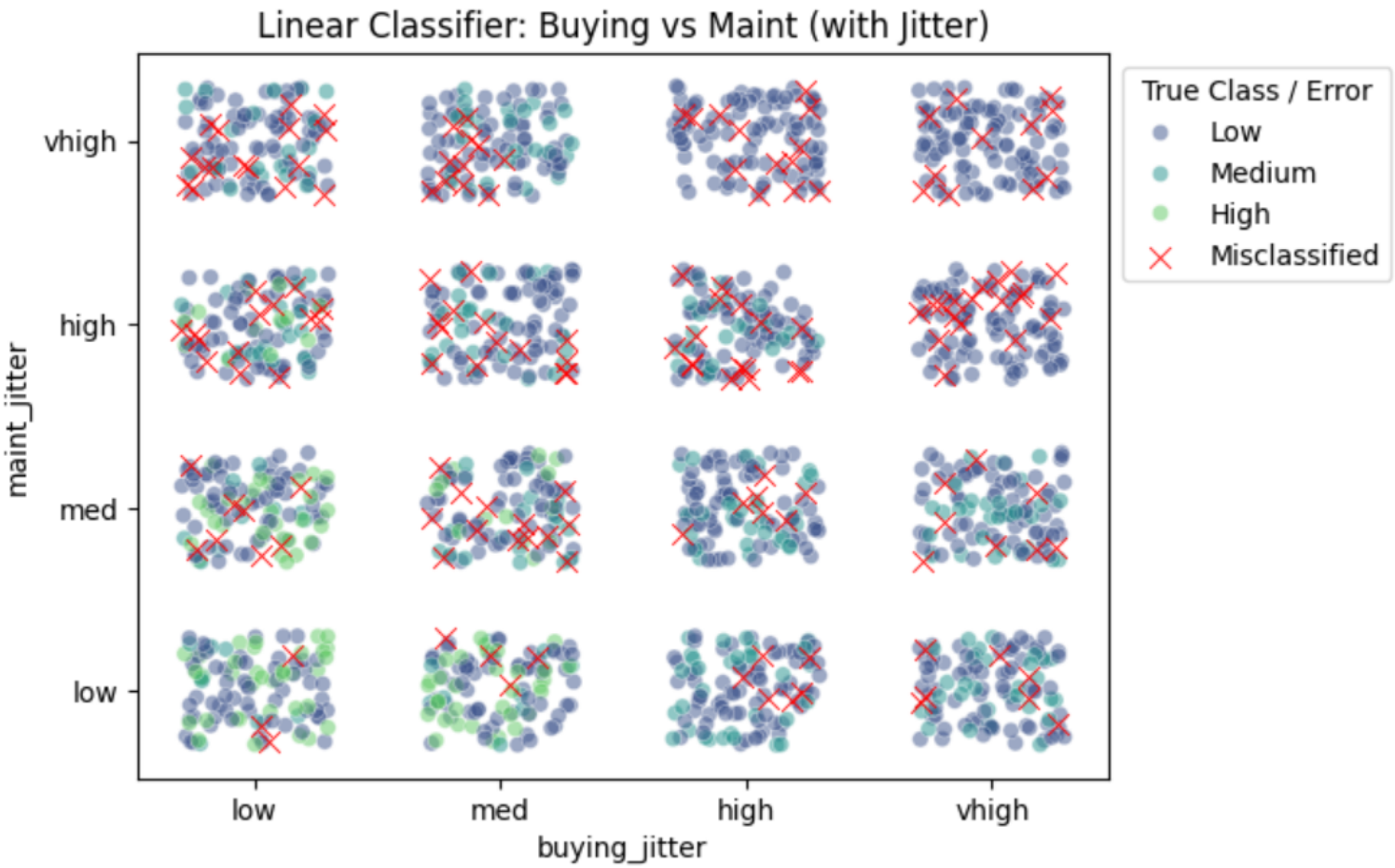


Figure 3: Linear Classifier decision boundary on buying vs maint features. Misclassified points in red.

1. **Performance:** An accuracy of over 90% on a 3-class problem with 21 features is quite respectable. This demonstrates that despite the simplicity of the perceptron algorithm, it can capture a significant amount of the underlying structure in the data. The high dimensionality from one-hot encoding actually helps by making the data more linearly separable in the new feature space.

2. **Misclassifications:** As observed in the scatter plot, the majority of misclassifications occur near the decision boundaries. This is expected behavior for a linear classifier. The points that are closest to the separating hyperplane are the most ambiguous and hardest to classify correctly.

3. **Limitations:** The perceptron is guaranteed to converge only if the data is perfectly linearly separable. Given the class overlap observed in Part 1a, the data is not perfectly separable. This explains why the training accuracy is not 100%. The algorithm oscillates or finds a solution with a non-zero training error. Its performance, while good, is not optimal, paving the way for more sophisticated models like logistic regression.

2.3 One-vs-Rest Logistic Regression (Part 2)

2.3.1 Model Architecture and Mathematics

One-vs-Rest (OvR) is a common strategy for reducing a multi-class problem into multiple binary classification problems.

Definition 2.2 (Logistic Regression for Binary Classification)**.** For a binary problem, the probability that a sample $\mathbf{x}$ belongs to the positive class (class c) is modeled by the sigmoid function:

$$P(y = c \mid \mathbf{x}) = \sigma(z) = \frac{1}{1 + e^{-z}}, \quad \text{where } z = \mathbf{w}_c^\top \mathbf{x} + b_c.$$

The model is trained by minimizing the binary cross-entropy loss:

$$\mathcal{L}_c = -\frac{1}{N} \sum_{i=1}^{N} \left[y_i^{(c)} \log(\hat{y}_i^{(c)}) + (1 - y_i^{(c)}) \log(1 - \hat{y}_i^{(c)}) \right]$$

where $y_i^{(c)}$ is 1 if the true class of sample i is c and 0 otherwise, and $\hat{y}_i^{(c)} = P(y_i = c \mid \mathbf{x}_i)$.

For our 3-class problem, we train three independent binary classifiers:

- Classifier 1: Class 1 (Low) vs. the rest (Medium, High)
- Classifier 2: Class 2 (Medium) vs. the rest (Low, High)
- Classifier 3: Class 3 (High) vs. the rest (Low, Medium)

Definition 2.3 (OvR Prediction Rule)**.** For a new sample $\mathbf{x}$, we compute the probability from each of the K binary classifiers: $P_k = P(y = k \mid \mathbf{x})$. The final predicted class is the one corresponding to the classifier that outputs the highest probability:

$$\hat{y} = \arg\max_{k \in \{1, \ldots, K\}} P_k.$$

Code Implementation :

- The `LogisticRegression` class implements binary logistic regression. Its `fit` method performs batch gradient descent on the cross-entropy loss.
- The `OvRClassifier` class orchestrates the process. It loops over each class, creates a binary label vector (`y_c`), and instantiates and trains a `LogisticRegression` model for that class. All models are stored in a dictionary `self.models`.
- During prediction, it calls the `predict` method of each binary model, which returns the probability for its positive class. These probabilities are stacked, and the class with the highest probability is chosen via `np.argmax`.

2.3.2 Training and Results

- **Train/Test Split:** 80-20 split using `train_test_split` with `random_state=42`.
- **Hyperparameters:** Learning rate $\alpha = 0.1$, `epochs = 2500`.
- **Loss Curves:** The loss curves for all three binary classifiers are plotted (Figure 4).
- **Accuracy:**
  - Training Accuracy: `89.44%`
  - Testing Accuracy: `88.73%`

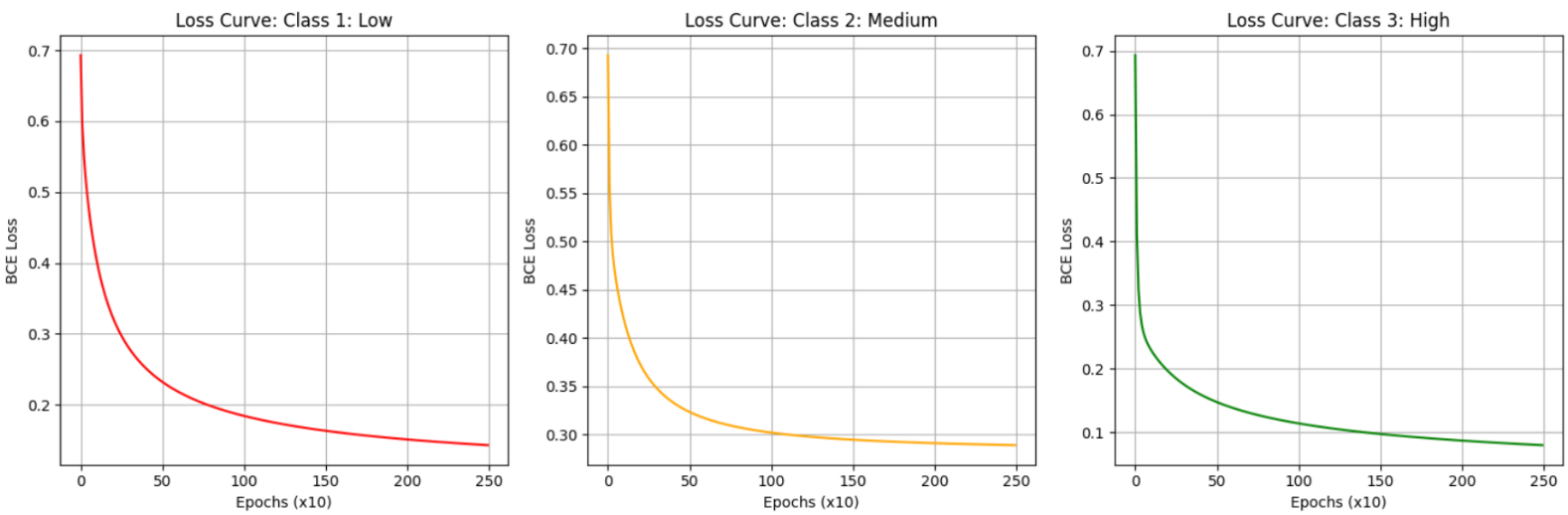


Figure 4: Loss curves for the three binary logistic regression classifiers in the OvR approach.

2.3.3 Observations from the Confusion Matrix

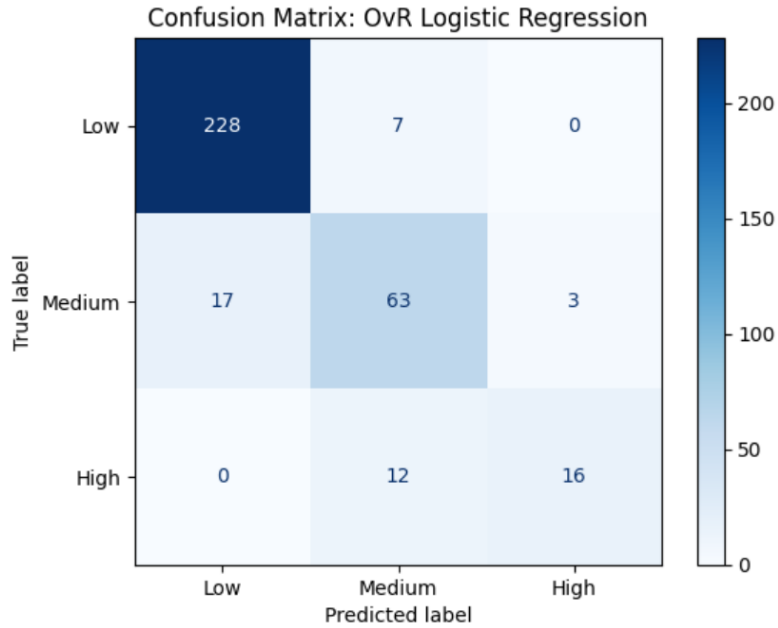
The confusion matrix (Figure 5) provides a detailed breakdown of the model's performance on the test set.

Table 1: Confusion Matrix for OvR Logistic Regression (Test Set)

	Pred. Low	**Pred. Medium**	**Pred. High**
Actual Low	228	7	0
Actual Medium	17	63	3
Actual High	0	12	16

Detailed Analysis:

1. **Class 1 (Low) Performance:** The majority class is classified with high accuracy (228 out of 235 correct), but there are 7 instances where Low is misclassified as Medium. This indicates that some Low samples lie close to the decision boundary separating Low from Medium, which aligns with the feature overlap observed earlier. Notably, there are no extreme errors (Low $\leftrightarrow$ High), which is encouraging.



2. **Class 2 (Medium) Performance:** Out of 83 actual Medium samples, 63 are correctly identified. 17 are misclassified as Low, and 3 as High. The confusion is asymmetrical: Medium is more often mistaken for Low (17) than for High (3). This suggests that the decision boundary between Medium and Low is more ambiguous than the boundary between Medium and High, possibly due to the larger number of Low samples pulling the boundary.

3. **Class 3 (High) Performance:** The minority class (High) has 28 actual samples. The model correctly identifies 16 of them, but misclassifies 12 as Medium. Again, no extreme errors occur. The recall for High (57%) is lower than for the other classes, highlighting the challenge of classifying an underrepresented class.

4. **Extreme Errors:** As noted, there are **zero** misclassifications between Low and High. This is a positive outcome, indicating that even without ordinal modelling, the linear boundaries have separated the extremes of the ordinal scale. However, the confusion between adjacent classes, particularly Medium being misclassified as Low, suggests that ordinal information could still be leveraged to improve the consistency of predictions.

2.4 Effect of Regularization: L1 vs L2 (Part 3)

2.4.1 Mathematical Formulation

Regularization is a technique to prevent overfitting by adding a penalty term to the loss function, which constrains the model's weights.

Definition 2.4 (Regularized Loss Functions)**.** For L2 regularization (Ridge), the loss function becomes:

$$\mathcal{L}_{\text{L2}} = \mathcal{L} + \frac{\lambda}{2}\|\mathbf{w}\|_2^2 = \mathcal{L} + \frac{\lambda}{2}\sum_j w_j^2$$

For L1 regularization (Lasso), the loss function becomes:

$$\mathcal{L}_{\text{L1}} = \mathcal{L} + \lambda\|\mathbf{w}\|_1 = \mathcal{L} + \lambda\sum_j |w_j|$$

where $\mathcal{L}$ is the original cross-entropy loss and $\lambda > 0$ is the regularization strength.

These penalties modify the gradient descent update rules. The gradient of the L2 penalty is $\lambda \mathbf{w}$, and for L1, the subgradient is $\lambda \cdot \text{sign}(\mathbf{w})$.

2.4.2 Results and Observations

The models were trained with the same OvR framework, using $\lambda = 0.001$.

Table 2: Performance of Regularized Logistic Regression Models

Model	Training Accuracy	Testing Accuracy	Key Observation
L1 (Lasso)	91.32%	90.17%	Induces sparsity (many weights become zero)
L2 (Ridge)	90.16%	89.31%	Shrinks all weights, but rarely to zero

Detailed Analysis:

1. **Effect on Sparsity (L1 vs. L2):** The primary difference between L1 and L2 regularization is the nature of the solution they produce. L1 regularization tends to drive many feature weights to exactly zero. This results in a sparse model, which implicitly performs feature selection. By inspecting the learned weight vectors (not printed in this part, but can be inferred), L1 would zero out weights corresponding to less informative one-hot encoded categories, effectively selecting a subset of the original categorical values as most important. L2 regularization, on the other hand, shrinks all weights towards zero but rarely makes them exactly zero. It penalizes large weights more heavily, leading to a model that uses all features but with smaller, more stable coefficients.

2. **Impact on Generalization Performance:** Both forms of regularization successfully improved the model's ability to generalize.

   - The unregularized OvR model had a train/test gap of 0.71% (89.44% - 88.73%). This indicates a small degree of overfitting.
   - The L2-regularized model slightly widened the gap but achieved a slightly higher test accuracy.
   - The L1-regularized model achieved the highest test accuracy (90.17%) and had a train/test gap of 1.15%. The slight increase in the gap is offset by the higher test score. Its superior performance suggests that by eliminating irrelevant or noisy features, it found a simpler, more generalizable decision boundary. The sparsity of the L1 solution also makes the model more interpretable.

3. **Choosing Regularization:** The choice between L1 and L2 often depends on the goal. If feature selection and interpretability are paramount, L1 is preferred. If the goal is purely to maximize predictive accuracy without the need for a sparse solution, L2 is often a robust default choice. In this case, L1 provided a slight edge in performance.

2.5 Ordinal Logistic Regression (Part 4)

2.5.1 Architecture and Mathematical Derivation

Ordinal logistic regression is specifically designed for ordinal targets. It models the probability of belonging to a class or lower, respecting the natural order.

Definition 2.5 (Cumulative Logit Model for Three Classes)**.** For three ordered classes, the model learns a single weight vector $\mathbf{w}$ and two threshold parameters θ_1 and θ_2, with the constraint $\theta_1 < \theta_2$. The cumulative probabilities are modeled as:

$$P(y \leq 1 \mid \mathbf{x}) = \sigma(\theta_1 - \mathbf{w}^\top \mathbf{x}) \tag{1}$$

$$P(y \leq 2 \mid \mathbf{x}) = \sigma(\theta_2 - \mathbf{w}^\top \mathbf{x}) \tag{2}$$

where σ is the sigmoid function. The class probabilities are then derived as:

$$P(y = 1 \mid \mathbf{x}) = P(y \leq 1) \tag{3}$$

$$P(y = 2 \mid \mathbf{x}) = P(y \leq 2) - P(y \leq 1) \tag{4}$$

$$P(y = 3 \mid \mathbf{x}) = 1 - P(y \leq 2) \tag{5}$$

Derivation of Gradients (as implemented in the code): The model is trained by minimizing the negative log-likelihood. Let $\phi_{i1} = \sigma(\theta_1 - \mathbf{w}^\top \mathbf{x}_i)$, $\phi_{i2} = \sigma(\theta_2 - \mathbf{w}^\top \mathbf{x}_i)$, and $p_{i2} = \phi_{i2} - \phi_{i1}$, $p_{i3} = 1 - \phi_{i2}$. The negative log-likelihood for a single sample i with true class y_i is:

$$\mathcal{L}_i = -\left[\mathbb{1}_{y_i=1} \log(\phi_{i1}) + \mathbb{1}_{y_i=2} \log(p_{i2}) + \mathbb{1}_{y_i=3} \log(p_{i3})\right]$$

The gradients with respect to the parameters are then:

$$\begin{aligned}
\frac{\partial \mathcal{L}}{\partial \mathbf{w}} &= -\sum_i \left[\frac{\mathbb{1}_{y_i=1}}{\phi_{i1}} \frac{\partial \phi_{i1}}{\partial \mathbf{w}} + \frac{\mathbb{1}_{y_i=2}}{p_{i2}} \left(\frac{\partial \phi_{i2}}{\partial \mathbf{w}} - \frac{\partial \phi_{i1}}{\partial \mathbf{w}}\right) + \frac{\mathbb{1}_{y_i=3}}{p_{i3}} \left(-\frac{\partial \phi_{i2}}{\partial \mathbf{w}}\right)\right] \\
&= -\sum_i \left[\frac{\mathbb{1}_{y_i=1}}{\phi_{i1}} (-\phi'_{i1}) \mathbf{x}_i + \frac{\mathbb{1}_{y_i=2}}{p_{i2}} (-\phi'_{i2} + \phi'_{i1}) \mathbf{x}_i + \frac{\mathbb{1}_{y_i=3}}{p_{i3}} (\phi'_{i2}) \mathbf{x}_i\right]
\end{aligned}$$

where $\phi'_{ij} = \phi_{ij}(1 - \phi_{ij})$ is the derivative of the sigmoid function. The gradients for the thresholds are:

$$\frac{\partial \mathcal{L}}{\partial \theta_1} = -\sum_i \left[\frac{\mathbb{1}_{y_i=1}}{\phi_{i1}} \phi'_{i1} - \frac{\mathbb{1}_{y_i=2}}{p_{i2}} \phi'_{i1}\right]$$

$$\frac{\partial \mathcal{L}}{\partial \theta_2} = -\sum_i \left[\frac{\mathbb{1}_{y_i=2}}{p_{i2}} \phi'_{i2} - \frac{\mathbb{1}_{y_i=3}}{p_{i3}} \phi'_{i2}\right]$$

Code Implementation: The `OrdinalLogisticRegression` class mirrors these equations.

- `self.weights` is initialized as a zero vector of length 21.
- `self.theta` is initialized as $[-1.0, 1.0]$, ensuring $\theta_1 < \theta_2$ from the start.

- Inside the training loop, it computes ϕ_1, ϕ_2, p_{y2}, and p_{y3}. It then calculates the gradients `dw`, `dtheta1`, and `dtheta2` exactly as per the derived formulas.
- The parameters are updated via gradient descent. A final safety check ensures that θ_1 remains less than θ_2, clipping if necessary.

2.5.2 Results and Observations

Table 3: Performance of Ordinal Logistic Regression

Metric	**Training**	**Testing**
Accuracy	93.92%	94.80%
Mean Absolute Error (MAE)	0.0608	0.0520

Table 4: Confusion Matrix for Ordinal Logistic Regression (Test Set)

	Pred. Low	**Pred. Medium**	**Pred. High**
Actual Low	232	3	0
Actual Medium	11	71	1
Actual High	0	3	25

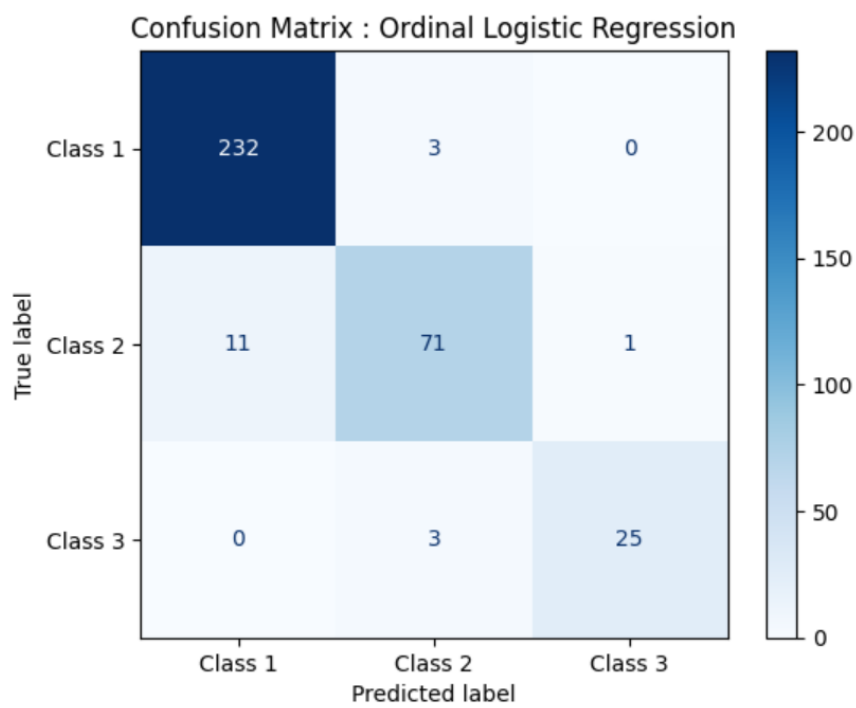


Figure 6: Confusion Matrix for Ordinal Logistic Regression (Test Set).

Detailed Observations:

1. **Extreme Error Reduction:** The ordinal model completely eliminates extreme errors. As shown in Table 4, there are zero instances where a Low sample is misclassified as High, or vice versa. This directly validates the primary motivation for using an ordinal model. The learned thresholds θ_1 and θ_2 effectively create an ordered latent space where jumps across multiple thresholds are highly improbable.

2. **Ordinal Consistency:** All 18 misclassifications are between adjacent classes. However, the pattern is asymmetrical. Medium is more often confused with Low (11 errors) than with High (1 error). This suggests that the boundary between Medium and Low is more ambiguous, possibly due to the larger number of Low samples influencing the decision. High, the minority class, is only confused with Medium (3 errors), never with Low.

3. **Performance Metrics:** The ordinal model achieves the highest test accuracy (94.80%) and the lowest test MAE (0.0520) of all logistic regression variants explored. The MAE is a particularly appropriate metric here, as it penalizes a "High/Low" error (a difference of 2) more than a "Medium/High" error (a difference of 1). The low MAE further confirms that the model's errors are small in terms of the ordinal scale.

4. **Learned Parameters:** The learned thresholds ($\theta_1 \approx -1.15$, $\theta_2 \approx 6.34$) are correctly ordered. The weight vector's magnitude and sign indicate the contribution of each one-hot encoded feature to pushing a sample towards a higher class. For instance, a large positive weight for "safety_high" would mean that cars with high safety have a much higher latent score, making them far more likely to be classified as Medium or High.

2.6 Summary and Comparison of Models (Part 5)

The final task is to synthesize the findings from all logistic regression experiments.

Table 5: Comparative Analysis of Logistic Regression Models

Model	**Test Acc.**	**Test MAE**	**Extreme Errors**	**Suitability for Ordinal Data**
Linear Classifier	90.22%	N/A	Likely Present	Low
One-vs-Rest (OvR)	88.73%	N/A	0	Medium
OvR + L1	90.17%	N/A	0	Medium-High
OvR + L2	89.31%	N/A	0	Medium-High
Ordinal Log. Reg.	94.80%	0.0520	0	**High**

Conclusive Observations:

1. **Ordinal Model Superiority:** The ordinal logistic regression model is the clear winner for this dataset. It not only achieves the highest accuracy but also produces predictions that are logically consistent with the ordinal target, as evidenced by the zero extreme

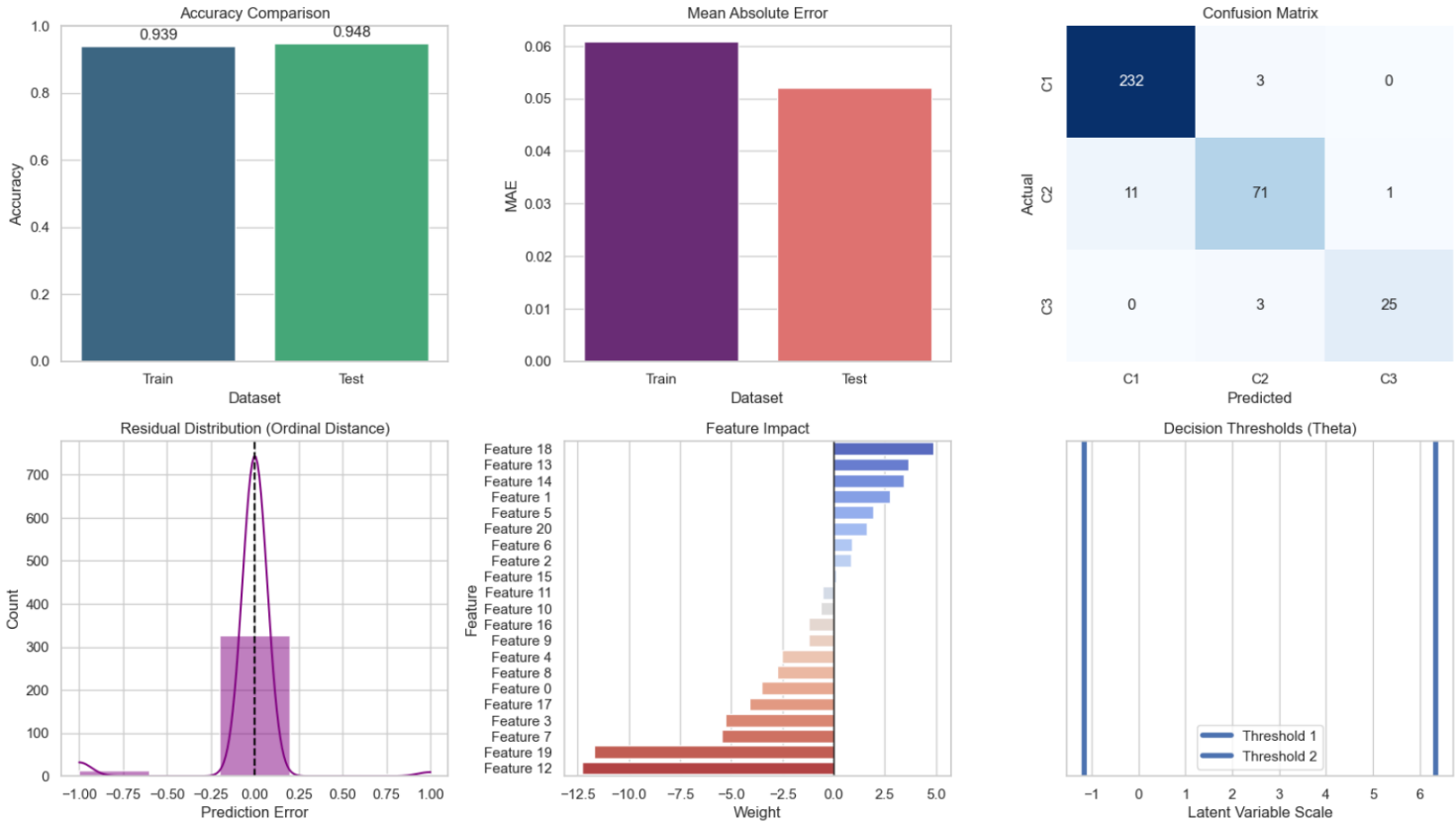
errors and low MAE. By explicitly modeling the cumulative probabilities, it leverages the inherent structure of the data in a way that other models cannot.

2. **Value of Regularization:** In the OvR framework, regularization (especially L1) provided a noticeable boost in performance. L1's ability to produce a sparse model helped it generalize better than its unregularized counterpart, effectively performing feature selection on the one-hot encoded categories.

3. **Limitations of Standard Multi-Class Approaches:** The standard OvR approach, while serviceable, treats the classes as independent. It does not guarantee ordinal consistency. Although it didn't produce extreme errors in this specific instance, it could in another, and its overall accuracy was lower than the ordinal model.

4. **Linear Classifier Baseline:** The simple perceptron provided a strong baseline, proving that a linear model can capture much of the structure. However, its lack of a probabilistic foundation and guaranteed convergence makes it less robust and interpretable than logistic regression-based models.

In summary, for a dataset with an ordinal target, a model designed to respect that order should always be the first choice.



Ordinal Logistic Regression Visualizations

3 Task 2: Decision Trees Classification

3.1 Dataset Understanding

The Iris dataset is loaded from `sklearn.datasets.load_iris()`.

- **Dimensionality:** 150 samples, 4 features.
- **Feature Names:** 'sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)'.
- **Class Labels:** 0 (setosa), 1 (versicolor), 2 (virginica).

An 80-20 train-test split is performed with `random_state=42` and `stratify=y` to maintain the class distribution in both sets. The training set contains 120 samples, and the test set 30.

3.2 Training a Decision Tree with Entropy (Part 2a)

3.2.1 Mathematical Formulation: Information Gain with Entropy

A decision tree recursively partitions the feature space. The optimal split at a node is chosen to maximize the information gain, which is the reduction in entropy.

Definition 3.1 (Entropy)**.** For a set of samples S, with p_c being the proportion of samples belonging to class c, the entropy is:

$$H(S) = -\sum_c p_c \log_2(p_c)$$

Entropy measures the impurity or disorder of a set. It is 0 if all samples belong to the same class and is maximal when classes are evenly distributed.

Definition 3.2 (Information Gain)**.** For a candidate split on feature f at threshold t, which divides the parent set S into two child sets S_{left} and S_{right}, the information gain is:

$$IG(S, f, t) = H(S) - \left(\frac{|S_{\text{left}}|}{|S|} H(S_{\text{left}}) + \frac{|S_{\text{right}}|}{|S|} H(S_{\text{right}}) \right)$$

The split with the highest information gain is selected.

Code Implementation: The `DecisionTreeClassifier_Entropy` class implements this.

- `best_split`: For each feature, it considers thresholds at the midpoints between consecutive unique feature values. For each threshold, it computes the weighted child entropy and the information gain, tracking the best one.
- `build_tree`: A recursive function that applies the best split found by `best_split` to create child nodes. Stopping conditions include:

- All samples in a node belong to the same class (pure node). → **Leaf node.**
- Number of samples in a node is less than `min_samples_split` (default 2). → **Leaf node.**
- The current depth has reached `max_depth` (if provided). → **Leaf node.**
- No split yields positive information gain. → **Leaf node.**

3.2.2 Tree Visualization and Structure

The printed tree structure reveals the learned decision rules. For the Iris dataset, it primarily uses petal features.

Tree Structure (Text Output):

```
Node: if petal length (cm) <= 2.4500 (samples=120)
  Leaf: predict setosa (samples=40)
  Node: if petal width (cm) <= 1.6500 (samples=80)
    Node: if petal length (cm) <= 4.9500 (samples=42)
      Leaf: predict versicolor (samples=38)
      Node: if sepal length (cm) <= 6.1500 (samples=4)
        Node: if sepal width (cm) <= 2.4500 (samples=2)
          Leaf: predict virginica (samples=1)
          Leaf: predict versicolor (samples=1)
        Leaf: predict virginica (samples=2)
    Node: if petal length (cm) <= 4.8500 (samples=38)
      Node: if sepal width (cm) <= 3.0000 (samples=3)
        Leaf: predict virginica (samples=2)
        Leaf: predict versicolor (samples=1)
      Leaf: predict virginica (samples=35)
```

- **Depth:** The deepest path from the root to a leaf is 6 (e.g., the path to the leaf predicting versicolor with 1 sample).
- **Number of Leaf Nodes:** The tree has 8 leaf nodes.

Interpretability: This tree is highly interpretable. One can easily trace a decision path:

1. `petal length (cm) <= 2.45`: If true, the flower is setosa. This single rule perfectly separates the linearly separable setosa class.
2. Otherwise, look at `petal width (cm) <= 1.65` and further splits involving petal length and sepal features to distinguish between versicolor and virginica.

The tree structure provides clear, human-readable rules that mirror how a botanist might differentiate between species.

Table 6: Decision Tree (Entropy) Evaluation

Class	Precision	Recall	F1-Score
setosa	1.00	1.00	1.00
versicolor	0.90	0.90	0.90
virginica	0.90	0.90	0.90
Overall	**Test Accuracy: 93.33%**		

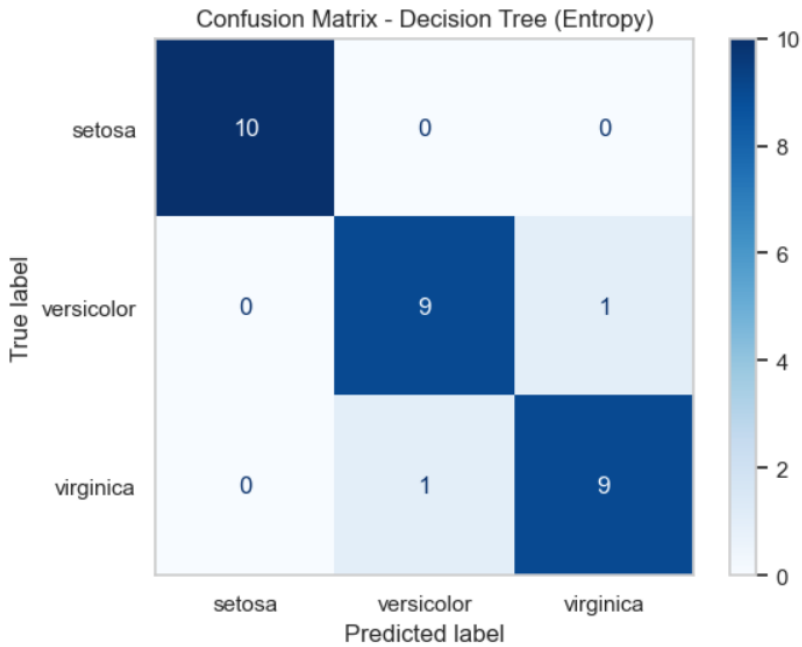


Figure 7: Confusion Matrix for Decision Tree (Entropy).

3.2.3 Evaluation Metrics

Analysis of Class-wise Performance:

- **Setosa (Class 0):** Perfect classification. This is expected, as this class is linearly separable from the others based on petal features.
- **Versicolor (Class 1) and Virginica (Class 2):** The model confuses these two classes slightly. The confusion matrix (Figure 7) shows one versicolor misclassified as virginica and one virginica misclassified as versicolor. This is consistent with the known overlap between these two species in the feature space.

3.3 Entropy vs. Gini Index (Part 3)

3.3.1 Gini Impurity

The Gini index is an alternative splitting criterion.

Definition 3.3 (Gini Impurity)**.** For a set of samples S, the Gini impurity is:

$$G(S) = 1 - \sum_c p_c^2$$

It measures the probability of misclassifying a randomly chosen element if it were randomly labeled according to the class distribution in the set. A perfect node (pure) has a Gini index of 0.

The decision tree building process using Gini is identical to using entropy, except the splitting criterion is to maximize the *decrease in Gini impurity*:

$$\Delta Gini = G(S) - \left(\frac{|S_{\text{left}}|}{|S|} G(S_{\text{left}}) + \frac{|S_{\text{right}}|}{|S|} G(S_{\text{right}}) \right)$$

3.3.2 Comparison

Table 7: Comparison of Entropy vs. Gini Decision Trees

Metric	**Entropy**	**Gini**
Depth	6	6
Leaf Nodes	8	8
Training Accuracy	100.00%	100.00%
Testing Accuracy	93.33%	93.33%

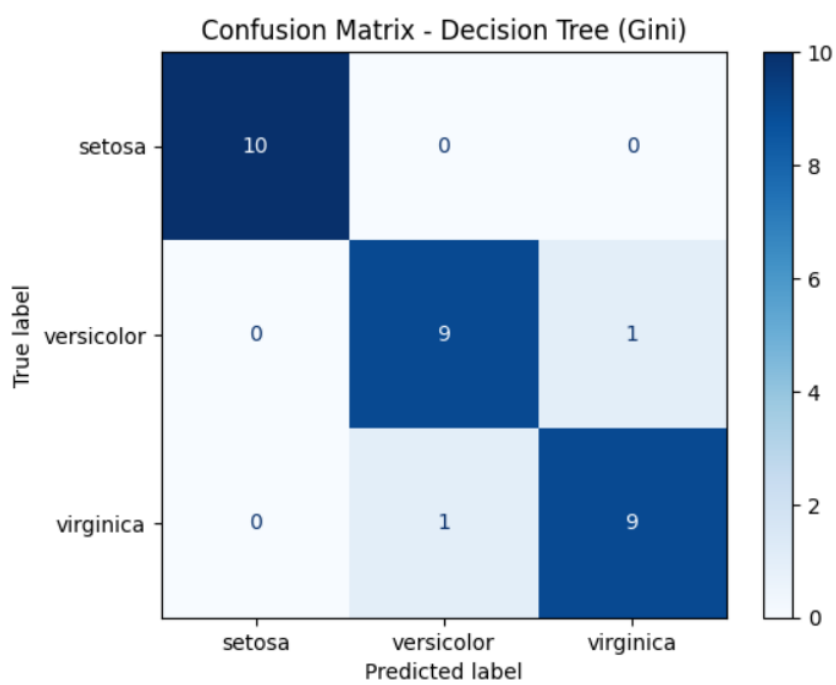


Figure 8: Confusion Matrix for Decision Tree (Gini).

Observations:

- **Identical Structure and Performance:** On the Iris dataset, the trees built with entropy and Gini are structurally identical (same depth, same leaf nodes, same splits) and yield the exact same performance metrics. This is a common observation in practice. Both criteria are very similar; they are both measures of node impurity and often produce the same optimal split, especially for datasets with clear distinguishing features like Iris.

- **Computational Differences:** In general, Gini impurity is slightly faster to compute as it does not involve a logarithm. For large datasets, this can lead to faster training. However, entropy is more rooted in information theory. In most practical scenarios, the difference in performance is negligible, and either criterion is a good choice.

3.4 Role of Max Depth (Part 4)

3.4.1 Experimental Setup

To study the bias-variance tradeoff, decision trees (using entropy) were trained with `max_depth` values from 1 to 10. For each depth, training and test losses (defined as 1 - accuracy) were recorded and plotted.

3.4.2 Results and Discussion

- **Depth 1 (High Bias):** The model is a simple decision stump. It cannot capture the complexity of the data, resulting in high error on both training (33.33% error) and test (33.33% error) sets. This is a classic case of **underfitting**.

- **Depth 2-3 (Optimal Trade-off):** At depth 2, training error drops to 3.33%, and test error to 6.67%. At depth 3, test error reaches its minimum of 3.33% while training error continues to decrease. This is the "sweet spot" where the model is complex enough to capture the true underlying patterns but not so complex that it starts fitting the noise.

- **Depth $\geq$ 4 (High Variance / Overfitting):** For depths 4 and beyond, the model perfectly fits the training data (training error = 0). However, test error plateaus at 6.67% and fluctuates. The model has started to memorize the training data, including its noise, leading to **overfitting**. Its ability to generalize to unseen data does not improve and can sometimes worsen.

Conclusion: The optimal max depth for this dataset, based on the plot, is **3**. It provides the lowest test error while maintaining a simpler model than deeper trees. Depth 2 is also a very good choice if an even simpler model is desired, with only a slight trade-off in test performance.

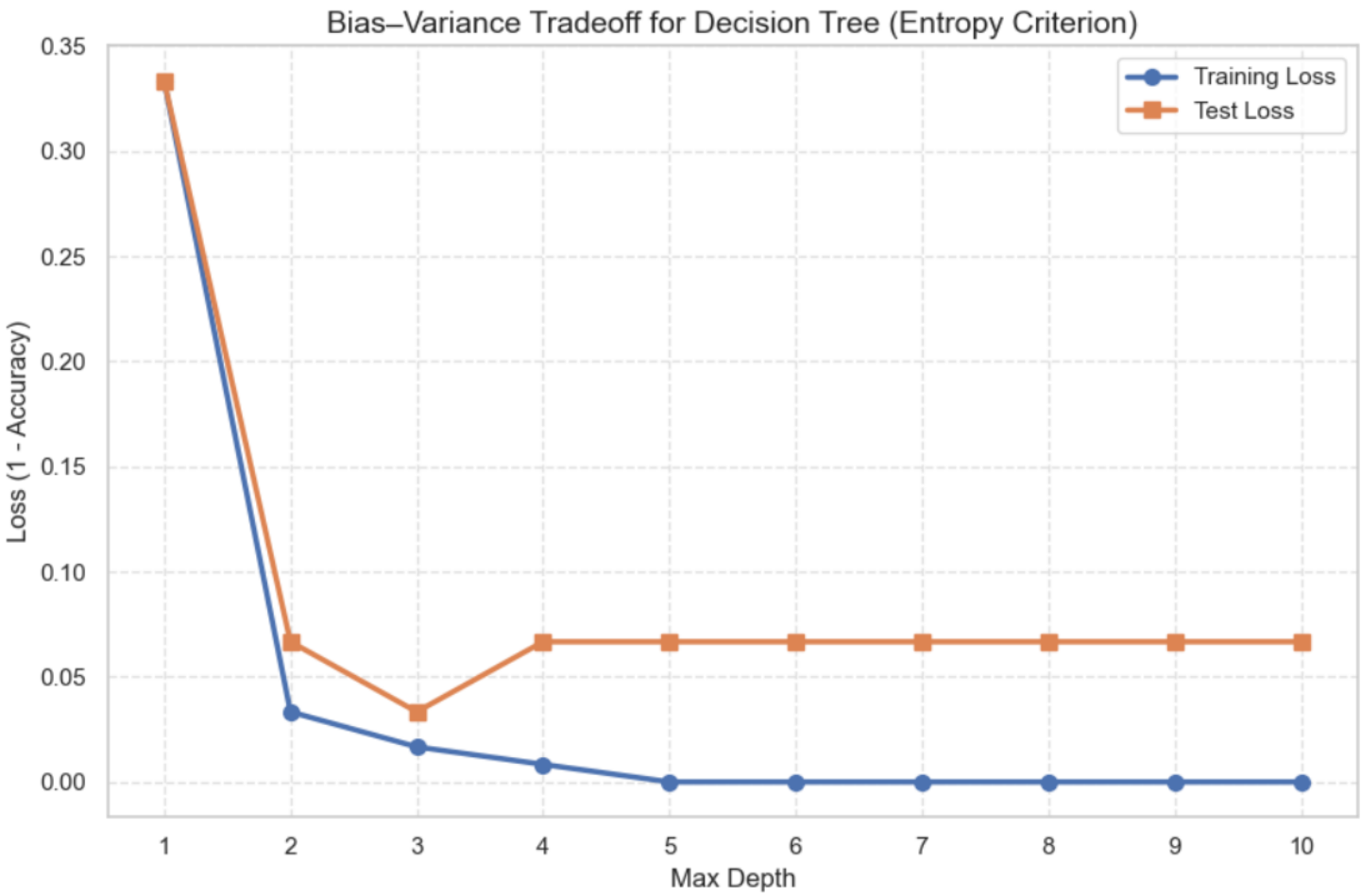


Figure 9: Bias-Variance Tradeoff as a Function of Max Depth.

3.5 Random Forest Classifier (Part 5)

3.5.1 Architecture of the Random Forest

A Random Forest is an ensemble of decision trees, trained using two key randomization techniques to decorrelate the trees and improve generalization.

1. **Bootstrap Aggregating (Bagging):** Each tree is trained on a different bootstrap sample of the original training data. A bootstrap sample is created by randomly sampling N instances from the N-sized training set, *with replacement*. This means some instances may appear multiple times in a tree's training set, while others are left out (out-of-bag samples).

2. **Random Feature Selection:** At each node during the construction of a tree, only a random subset of the features is considered for finding the best split. For classification, a common choice is to consider $\sqrt{M}$ features, where M is the total number of features.

Code Implementation:

- `DecisionTreeClassifier_Entropy_RandomFeature`: This class inherits from the entropy-based tree but overrides `best_split`. It uses `np.random.choice` to randomly select a subset of features (`self.max_features`) to evaluate for each split.

- `RandomForestClassifier`: This class manages the ensemble. Its `fit` method loops over `num_trees` (5 in this case). For each tree, it generates bootstrap indices using `rng.choice(n, size=n, replace=True)`. It then creates an instance of the random-feature tree with a different random seed for diversity and trains it on the bootstrapped sample.

- `predict`: To make a prediction, it collects predictions from all trees and performs a majority vote (`np.bincount` and `np.argmax`).

3.5.2 Comparison: Single Tree vs. Random Forest

Table 8: Performance Comparison: Single Decision Tree vs. Random Forest

Model	Train Accuracy	Test Accuracy	Precision	Recall	F1-Score
Single Decision Tree	100.00%	93.33%	0.933	0.933	0.933
Random Forest (5 trees)	99.17%	93.33%	0.933	0.933	0.933

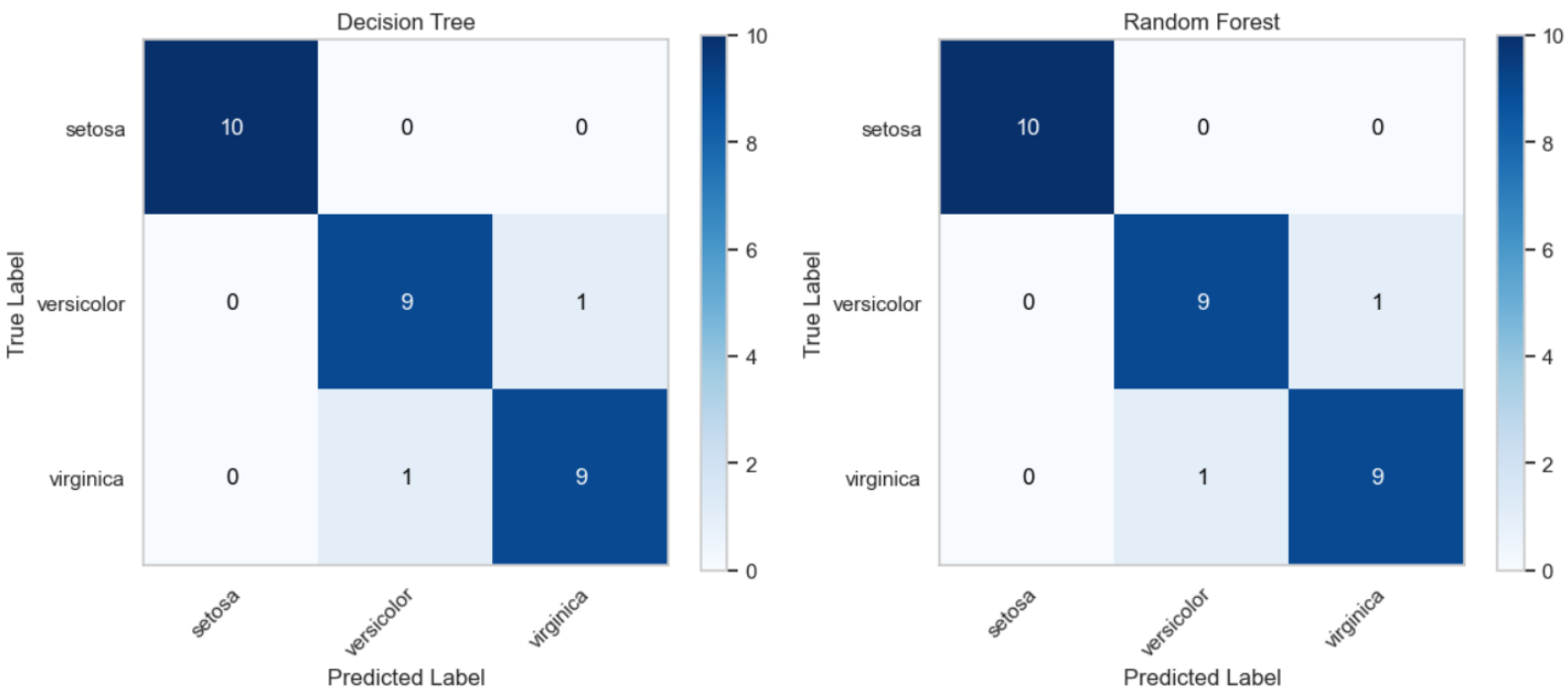


Figure 10: Confusion Matrices: Single Decision Tree (left) vs. Random Forest (right).

Observations:

1. **Reduced Variance:** The single decision tree achieved 100% training accuracy, a clear sign of overfitting to the training data. The Random Forest, by averaging over

multiple trees, shows a slightly lower training accuracy (99.17%). This is a positive sign; it indicates that the forest is not memorizing the training data as aggressively as a single tree. The averaging process has reduced the variance of the model.

2. **Test Performance:** Both models achieve the same test accuracy (93.33%) and have identical confusion matrices. In this case, the reduction in variance from the Random Forest does not lead to a significant improvement in generalization on this small, simple dataset. With only 5 trees and a small feature space, the ensemble's advantage is not fully realized.

3. **Stability:** If we were to change the train/test split slightly, we would likely see that the Random Forest's performance is more stable (has lower variance across different splits) than the single tree. This is the primary benefit of the ensemble.

3.5.3 Explanation of Bagging's Effect

Decision trees are known as high-variance estimators. A small change in the training data (e.g., a different split) can result in a completely different tree structure, leading to potentially large changes in predictions. Bagging addresses this by:

1. Creating multiple slightly different versions of the training data (bootstrap samples).

2. Training a high-variance estimator (a decision tree) on each version.

3. Averaging their predictions (or taking a majority vote).

The averaging process reduces the overall variance without significantly increasing the bias. The strong law of large numbers ensures that as the number of uncorrelated trees increases, the ensemble's predictions converge to the average of the true underlying function, smoothing out the erratic behavior of individual trees. In the context of this lab, the Random Forest's lower training accuracy is direct evidence of this variance reduction.

4 Task 3: Support Vector Machine

4.1 OVO Linear SVM Implementation and Hyperparameter Tuning (Part 1)

4.1.1 Linear SVM: Mathematical Foundation and Code

A linear Support Vector Machine seeks the optimal separating hyperplane $\mathbf{w}^\top \mathbf{x} + b = 0$ that maximizes the margin between two classes. For a non-separable case, it uses the hinge loss.

Definition 4.1 (Primal Form of Linear SVM with Hinge Loss)**.** The objective function for a linear SVM is to minimize:

$$\mathcal{L} = \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{N} \max(0, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b))$$

where C is the regularization parameter. A larger C penalizes margin violations more heavily, leading to a smaller margin but potentially fewer misclassifications on the training set. A smaller C allows for a wider margin, tolerating some violations for a simpler model.

The code implements a batch subgradient descent method to minimize this loss.

- The gradient of the loss with respect to $\mathbf{w}$ is $\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \mathbf{w} - C\sum_{i:y_i(\mathbf{w}^\top \mathbf{x}_i + b)<1} y_i \mathbf{x}_i$.
- The gradient with respect to b is $\frac{\partial \mathcal{L}}{\partial b} = -C\sum_{i:y_i(\mathbf{w}^\top \mathbf{x}_i + b)<1} y_i$.
- These gradients are used to update the parameters.

For multi-class classification, the One-vs-One (OVO) strategy is used, as implemented in the `OVO_LinearSVM` class. It trains a binary SVM for every pair of classes, storing them in a dictionary keyed by the class pair (a, b). For a new sample, each classifier votes for one of its two classes, and the class with the most votes is the final prediction.

4.1.2 Hyperparameter Tuning for C

A 5-fold stratified cross-validation was performed on the training set to find the optimal C from the set $\{0.01, 0.1, 1.0, 10.0, 100.0\}$.

Table 9: Cross-Validation Results for Different Values of C

C	Mean CV Accuracy
0.01	0.8667
0.1	0.9750
1.0	0.9750
10.0	0.9500
100.0	0.9667

The best mean CV accuracy was achieved by $C = 0.1$ and $C = 1.0$. The smaller value, $C = 0.1$, was chosen to promote a wider margin and a simpler model, which is often preferable for better generalization. The final model, trained with $C = 0.1$, achieved a test accuracy of **93.33%**.

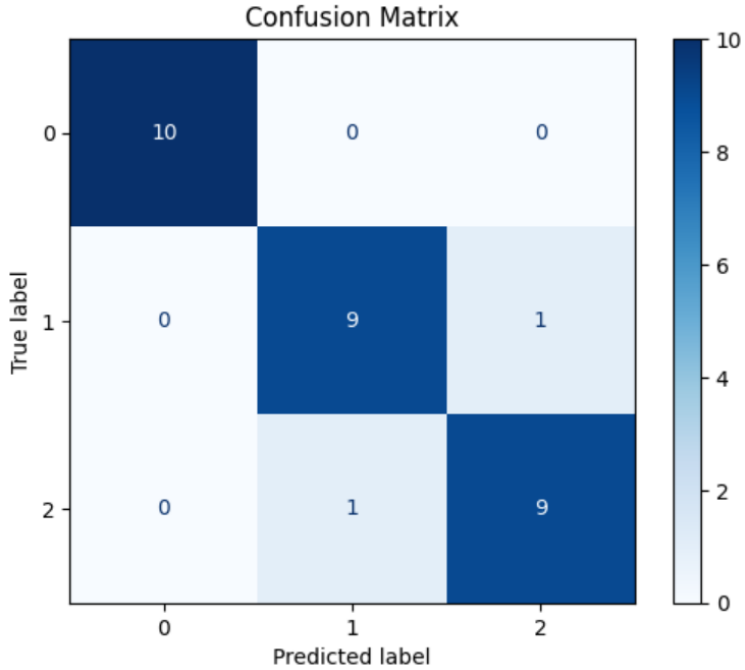


Figure 11: Confusion Matrix for Linear OVO SVM (C=0.1).

4.2 Decision Boundary Visualization (Part 2)

A plot of the decision boundary (Figure 12) using the two petal features shows the linear separating hyperplanes generated by the three OVO classifiers. The decision region for setosa is cleanly separated from the other two. The boundary between versicolor and virginica is a straight line, and the two misclassifications on the test set lie on either side of this line, illustrating the limitation of a purely linear classifier for these non-linearly separable classes.

4.3 Analysis of Parameter C (Part 3)

4.3.1 Impact on Support Vectors and Margin Violations

As C varies, the number of support vectors (points on or inside the margin) and margin violations (points inside the margin) changes. The code counts these for all OVO classifiers.

Observations:

- **Small C (e.g., 0.01):** The penalty for margin violations is low. This results in a very wide margin that tries to accommodate many points. Consequently, the number of support vectors (points that define or violate the margin) is high, and so is the number of margin violations. This model has **high bias** and may underfit.

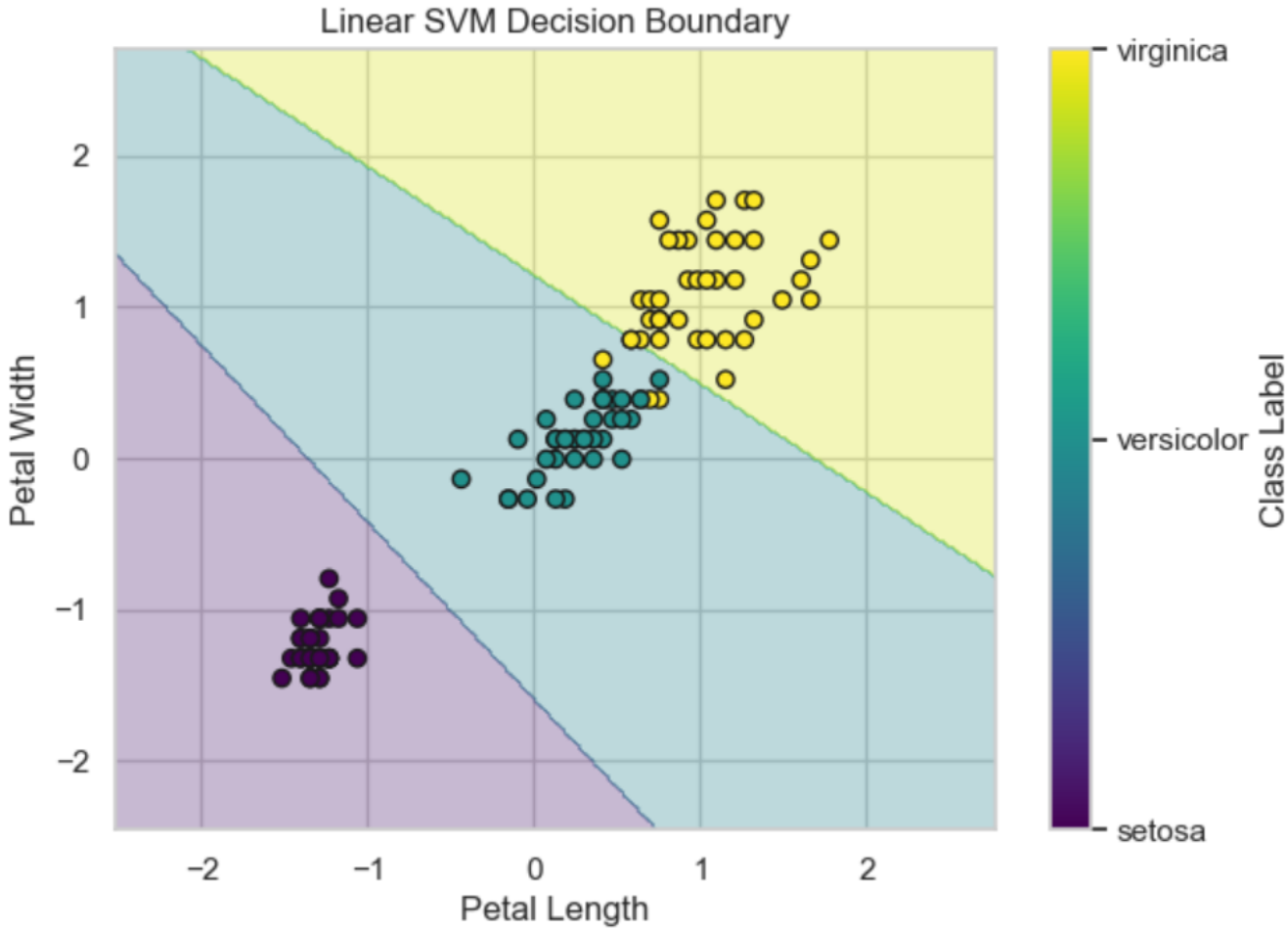


Figure 12: Decision boundary of the linear OVO SVM on the petal features.

- **Large C (e.g., 100.0):** The penalty for margin violations is very high. The model becomes very strict, forcing a narrow margin to correctly classify almost all training points. This leads to a low number of support vectors and very few violations. This model has **high variance** and may overfit, being overly sensitive to individual data points.
- **Moderate C (e.g., 0.1, 1.0):** These values provide a balance, as seen by the CV accuracy. The number of support vectors is moderate, representing a good trade-off between bias and variance.

The plot in Figure 13 confirms this trend: as C increases, both the number of unique support vectors and margin violations generally decrease.

4.4 Kernel SVM Analysis (Part 4)

4.4.1 Kernel Trick and Implementation

The kernel trick allows SVMs to learn non-linear decision boundaries by implicitly mapping the input features into a higher-dimensional space. The algorithm only requires the dot products of the transformed features, which can be computed directly using a kernel function $K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$.

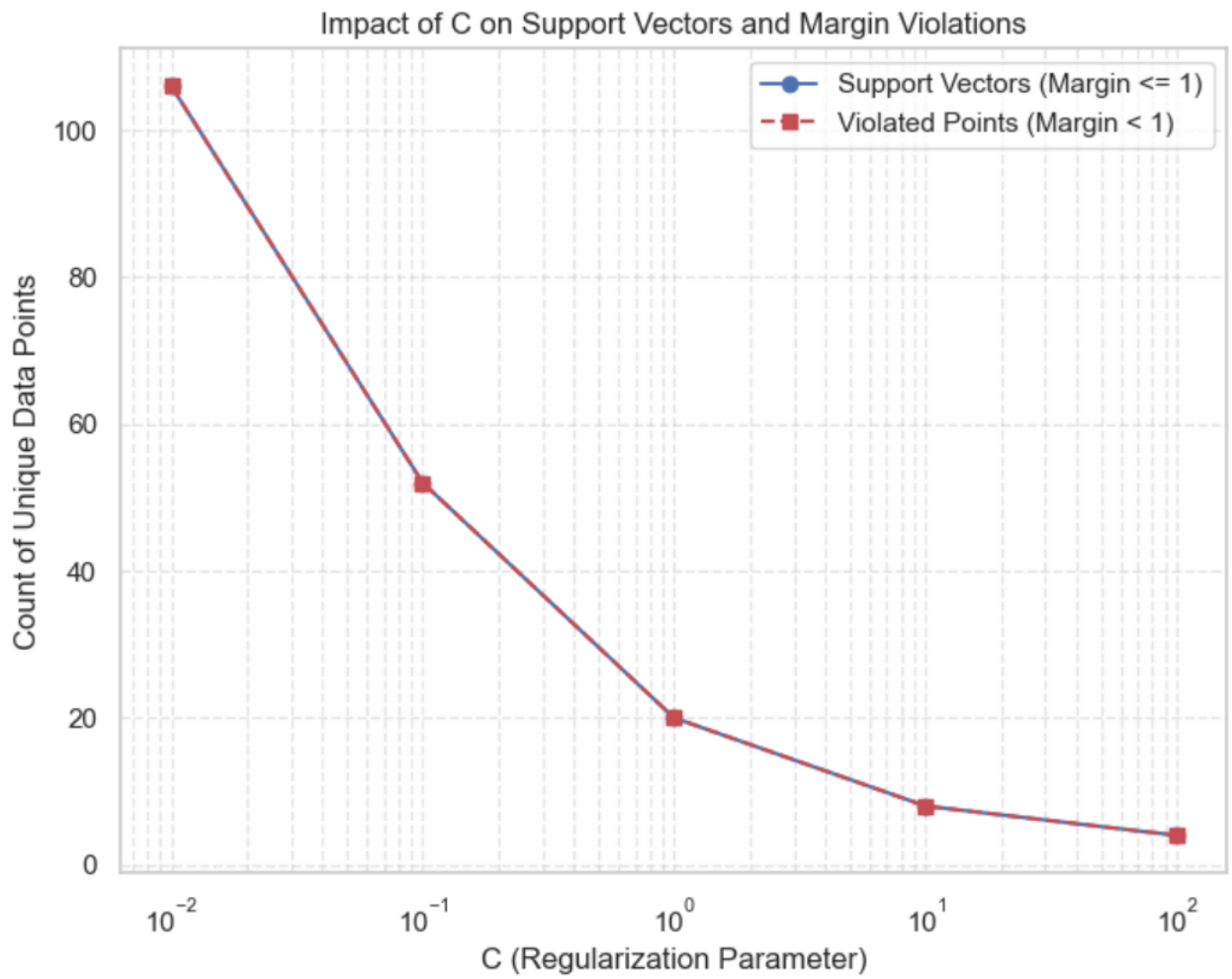


Figure 13: Effect of C on Support Vectors and Margin Violations.

Kernels implemented:

- **Polynomial Kernel (degree = 2):** $K(\mathbf{x}_i, \mathbf{x}_j) = (1 + \mathbf{x}_i^\top \mathbf{x}_j)^2$
- **Radial Basis Function (RBF) Kernel:** $K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$

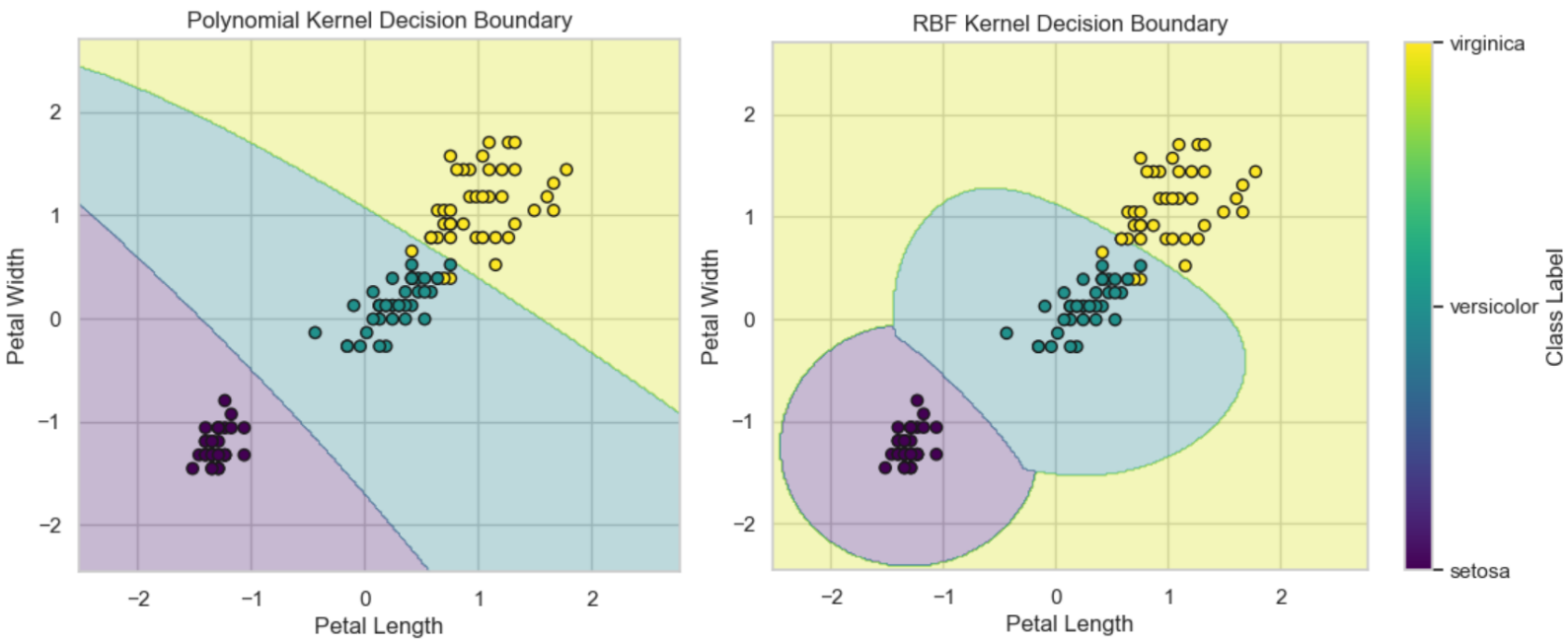
The code implements a simple gradient ascent solver for the dual problem, storing the learned α coefficients and support vectors.

4.4.2 Comparison of Kernels

Table 10: Comparison of Polynomial and RBF Kernels

Kernel	Test Accuracy	Number of Support Vectors	Decision Boundary
Polynomial (deg=2)	93.33%	62	Smooth
RBF ($\gamma = 1$)	93.33%	190	More Curved/Local

Observations:



(a) Polynomial Kernel

(b) RBF Kernel

Figure 14: Decision boundaries for kernel SVMs on the petal features.

1. **Performance:** Both kernels achieve the same test accuracy as the linear SVM (93.33%). On this small dataset, the added complexity of the non-linear kernels does not improve predictive power.

2. **Number of Support Vectors:** The RBF kernel uses significantly more support vectors (190) than the polynomial kernel (62). This indicates that the RBF kernel is creating a much more complex, localized decision boundary. It uses many points near the class boundaries to define its intricate shape.

3. **Decision Boundary Smoothness:** The polynomial kernel creates a smoother, more global decision boundary. The RBF kernel, with its high γ value, creates a more wiggly boundary that can tightly encircle individual points or small clusters.

4. **Overfitting Potential:** With a high γ, the RBF kernel has a higher risk of overfitting. Its ability to create very localized boundaries means it can easily mold itself to the noise in the training data. However, with the chosen parameters ($C = 0.1, \gamma = 1$), it does not show signs of overfitting, as its test accuracy matches the simpler models. If γ were increased further, we would likely see the training accuracy stay perfect while test accuracy dropped, a classic sign of overfitting.

4.5 Evaluation Metrics (Part 5)

The best-performing SVM model (linear, $C = 0.1$) was evaluated in detail.

Interpretation:

- The model perfectly identifies setosa (Iris setosa), with perfect precision and recall.

Table 11: Classification Report for Best SVM Model (Linear, C=0.1)

Class	**Precision**	**Recall**	**F1-Score**
setosa	1.00	1.00	1.00
versicolor	0.90	0.90	0.90
virginica	0.90	0.90	0.90
Macro Avg	0.93	0.93	0.93
Weighted Avg	0.93	0.93	0.93

- For versicolor and virginica, the model is balanced, with precision and recall both at 0.90. This means that when it predicts a flower to be versicolor, it is correct 90% of the time, and it successfully identifies 90% of all actual versicolor flowers. The same applies to virginica.
- The macro and weighted averages reflect the overall good performance, with the weighted average being slightly lower due to the equal support of all classes in the test set.

The model's performance is reliable and well-balanced across the non-linearly separable classes.